

OpenCorvus Architecture

1. System Architecture Overview

```

graph TB
    subgraph CLIENT["Client Layer"]
        direction LR
        OV["Overlay UI  
Solid.js + Tauri Desktop  
components / stores / services"]
        VS["VSCode Extension  
WebView + Native Panel  
Future"]
        CB["Channel Bots  
Slack / Discord / Telegram  
Teams / Feishu / DingTalk"]
    end

    subgraph API["API Gateway – Hono HTTP :7878"]
        direction LR
        MW["Auth Middleware  
CORS / Error Handler"]
        ROUTES["REST Routes  
/config /session /task /goal /plan  
/provider /executor /channel /control  
/auth /mcp /skill /permission"]
        SSE["SSE Event Stream  
GET /task/{id}/events  
message.part.delta  
reasoning.delta"]
        WS["WebSocket  
PTY terminal"]
    end

    subgraph CORE["Core Engine Layer"]
        SM["Session Manager  
CRUD / Message V2  
Prompt Pipeline  
Compaction / Revert"]
        ORC["Orchestrator  
Task State Machine  
Pipeline / Runtime  
Strategy / Checks"]
        CP["Control Plane  
Panel Requests  
Structured Output"]
        AG["Agent System  
Configs / Permissions  
Prompt Construction"]
        MEM["Memory"]
    end

```

```

Facts / Lessons
FTS5 Search
Scratchpad"]
    PROV["Provider System
15+ LLM Providers
OAuth + API Key
Streaming + Tools"]
    TOOLS["Tool Registry
70+ Tools:
bash / edit / read / write
glob / grep / webfetch / ..."]
    EXEC["Executors
opencode / codex
claude-code"]
    EVAL["Evaluator
Goal Checking
Quality Scoring"]
    DEL["Delivery
Git Branch / PR
Patch / Report"]
    end

    subgraph INFRA["Infrastructure Layer"]
        DB["SQLite + Drizzle ORM
25+ tables: session / message / part
orchestrator_* / memory / protocol"]
        BUS["Event Bus
Typed Pub/Sub
GlobalBus IPC"]
        CFG["Config System
remote → global → project
→ .opencorvus → managed"]
        PLG["Plugin System
Copilot / Codex / Anthropic
External npm / local"]
        SKL["Skills
SKILL.md format
Marketplace"]
        MCP["MCP
HTTP / SSE / stdio
Tool Integration"]
        AUTH["Auth Storage
~/.opencorvus/auth.json
OAuth / API Key"]
        SCHED["Scheduler
Cron / Event
Task Queue"]
    end

    OV -->|REST + SSE| API
    VS -->|REST + SSE| API
    CB -->|REST| API

    API --> CORE

```

```
CORE --> INFRA
```

```
style CLIENT fill:#1a2332,stroke:#58a6ff,color:#e6edf3
style API fill:#1a2332,stroke:#3fb950,color:#e6edf3
style CORE fill:#1a2332,stroke:#bc8cff,color:#e6edf3
style INFRA fill:#1a2332,stroke:#d29922,color:#e6edf3
style OV fill:#21262d,stroke:#58a6ff,color:#e6edf3
style VS fill:#21262d,stroke:#bc8cff,color:#e6edf3
style CB fill:#21262d,stroke:#3fb950,color:#e6edf3
```

2. Data Flow

```
flowchart TD
```

```
    INPUT["User Input  
Overlay / VSCode / Channel Bot"] --> GW["API Gateway  
REST / SSE"]
```

```
    GW --> CP["Control Plane  
panel requests"]
```

```
    GW --> ORC["Orchestrator  
task lifecycle"]
```

```
    CP --> SM["Session Manager  
prompt pipeline"]
```

```
    SM --> PROV["Provider System  
model resolution → LLM call"]
```

```
    ORC --> SPEC["Spec Agent  
PRD generation"]
```

```
    ORC --> GOAL["Goal Decomposer  
requirement breakdown"]
```

```
    ORC --> PLAN["Planner Agent  
execution plan"]
```

```
    ORC --> EXEC["Executor  
code execution"]
```

```
    ORC --> EVAL["Evaluator  
goal verification"]
```

```
    ORC --> DEL["Delivery  
PR / branch / report"]
```

```
    PROV --> SSE["SSE Event Stream"]
```

```
    DEL --> SSE
```

```
    SSE --> UI["Client UI  
real-time updates"]
```

```
style INPUT fill:#21262d,stroke:#58a6ff,color:#e6edf3
```

```
style GW fill:#21262d,stroke:#3fb950,color:#e6edf3
```

```
style CP fill:#21262d,stroke:#39d2c0,color:#e6edf3
```

```
style ORC fill:#21262d,stroke:#d29922,color:#e6edf3
```

```
style SSE fill:#21262d,stroke:#3fb950,color:#e6edf3
```

```
style UI fill:#21262d,stroke:#58a6ff,color:#e6edf3
```

3. Orchestrator — Task State Machine

```
stateDiagram-v2
    [*] --> queued
    queued --> spec_generating
    spec_generating --> goal_decomposing
    goal_decomposing --> planning
    planning --> planned
    planned --> running
    running --> blocked : needs user input
    blocked --> running : user responds
    running --> evaluating
    evaluating --> delivering : goals met
    evaluating --> planning : replan needed
    delivering --> completed
    running --> failed
    delivering --> failed
    [*] --> cancelled

    note right of spec_generating : Spec Agent generates PRD
    note right of goal_decomposing : LLM classifies + clusters requirements
    note right of planning : Planner Agent (8 tools, quality scoring)
    note right of running : opencode / codex / claude-code
    note right of evaluating : Goal checking + quality threshold
    note right of delivering : Git branch / PR / patch
```

Orchestrator Modules

Module	File	Description
Pipeline	pipeline.ts	Task queue processing
Runtime	runtime.ts	Execution runtime
Strategy	strategy.ts	Retry / replan decisions
Persist	persist.ts	State transitions
Checks	checks.ts	Goal verification
Interaction	interaction.ts	User interactions
Memory Bridge	memory-bridge.ts	Learning integration
Publisher	publisher.ts	Delivery publishing
Git	git.ts	Git operations
Agent Stream	agent-stream.ts	Agent message streaming

4. Provider System

```

graph LR
    subgraph Providers["Supported LLM Providers"]
        A[Anthropic]
        B[OpenAI]
        C[Azure]
        D[Google]
        E[Bedrock]
        F[GitHub Copilot]
        G[Groq]
        H[Mistral]
        I[DeepSeek]
        J[XAI]
        K[alibaba-cn]
        L[Fireworks]
        M[Together]
        N[Cerebras]
    end

    CFG["Config  
model: provider/model-id"] --> RESOLVE["Provider.defaultModel()"]
    RESOLVE --> GET["Provider.getModel()"]
    GET --> LANG["Provider.getLanguage()"]
    LANG --> LLM["LLM API Call  
streaming + tool calls"]

    style Providers fill:#1a2332,stroke:#bc8cff,color:#e6edf3

```

Model Resolution Priority

```
input.model → agent.model → Provider.defaultModel() → Config.get().model
```

Config Update Flow (UI Model Switch)

```

sequenceDiagram
    participant UI as Overlay UI
    participant API as PATCH /config
    participant CFG as Config.update()
    participant PRV as Provider.reset()
    participant LLM as Next LLM Call

    UI->>API: { model: "github-copilot/gemini-3-flash-preview" }
    API->>CFG: Write .opencorvus/opencorvus.jsonc
    CFG->>CFG: state.reset() + global.reset()
    API->>PRV: Clear cached providers
    Note over LLM: Next request picks up new model
    LLM->>PRV: Provider.defaultModel() → reads fresh config

```

5. VSCode Extension Architecture (Future)

```

graph TB
    subgraph VSCode["VSCode Extension Host"]
        ENTRY["Extension Entry  
activate() / deactivate()"]

        subgraph UI["UI Components"]
            WV["WebView Panel  
reuse overlay Solid.js"]
            SB["Sidebar Panel  
task list / agent status"]
            BAR["Status Bar  
connection / model / progress"]
        end

        subgraph EDITOR["Editor Integration"]
            CL["CodeLens  
inline agent actions"]
            DIAG["Diagnostics  
evaluation results"]
            DIFF["Diff Editor  
review delivery patches"]
            COMP["Inline Completion  
suggest from memory/plan"]
        end

        subgraph CMD["Command Palette"]
            C1["opencorvus.createTask"]
            C2["opencorvus.switchModel"]
            C3["opencorvus.viewBoard"]
            C4["opencorvus.openSettings"]
        end

        subgraph TERM["Terminal"]
            TP["Terminal Profile Provider"]
            PTY["PTY Integration  
live executor output"]
        end

        subgraph SVC["Service Layer"]
            SDK["OpenCorvusSDK  
from @opencorvus/sdk"]
            BRIDGE["Event Bridge  
SSE → EventEmitter"]
        end

        SVC -->|"REST + SSE  
(same protocol as Overlay)"| SERVER["OpenCorvus Core :7878"]

        style VSCode fill:#1a2332,stroke:#bc8cff,color:#e6edf3
        style SERVER fill:#21262d,stroke:#3fb950,color:#e6edf3
    
```

6. Package Dependency Graph

```
graph TD
  OC["opencorvus  
Core backend"] --> UTIL["util  
Shared utilities"]
  OC --> PLG["plugin  
Plugin infra"]
  OC --> SDK2["sdk  
Client SDK"]
  OC --> CC["channel-config  
Channel schemas"]
  OC --> CR["channel-runtime  
Channel handlers"]
  PLG --> UTIL

  OV["overlay  
Frontend UI"] -->|"REST + SSE"| OC
  SC["script  
Build tooling"] --> OC

  style OC fill:#21262d,stroke:#58a6ff,color:#e6edf3
  style OV fill:#21262d,stroke:#3fb950,color:#e6edf3
  style UTIL fill:#21262d,stroke:#d29922,color:#e6edf3
```

7. Database Schema (25+ tables)

Group	Tables
Session	session, message, part, todo, permission
Orchestrator	orchestrator_task, _run, _plan_version, _goal, _milestone, _spec_snapshot, _spec_item, _evaluation, _delivery, _artifact, _execution_session, _interaction_request
Control	control, control_timeline
Memory	memory, scratchpad, task_plan
System	protocol, project, workbench, share, cron, event, task_queue

8. Key Design Decisions

#	Decision	Rationale
1	Protocol-first	REST + SSE as universal interface — any UI connects identically
2	Compiled binary	bun build --compile → single executable with embedded UI assets
3	SQLite + Drizzle	Single-file DB, zero deps, event sourcing via protocol table
4	Plugin architecture	Auth, tools, hooks are pluggable (built-in + external npm/local)

#	Decision	Rationale
5	Multi-executor	opencode / codex / claude-code via adapter pattern
6	Config cascade	remote → global → project → .opencorvus → inline → managed

9. Config Priority Chain

```
graph LR
    R["Remote  
.well-known"] --> G["Global  
~/.config/opencorvus"]
    G --> C["Custom  
OPENCORVUS_CONFIG"]
    C --> P["Project  
opencorvus.json{,c}"]
    P --> D["".opencorvus/  
directory config"]
    D --> I["Inline  
ENV content"]
    I --> M["Managed  
Enterprise override"]

    style R fill:#21262d,stroke:#64748b,color:#e6edf3
    style M fill:#21262d,stroke:#f85149,color:#e6edf3
```

Low → High priority: 每一层覆盖前一层。 `.opencorvus/opencorvus.jsonc` 是 UI 配置的写入目标。